As mentioned, insertion starts with a search step; while for more basic trees, like *binary search trees*, an unsuccessful search returns the one and only node where the new item can be added, for SS-trees we have the same issue we briefly discussed in section 10.2 for R-trees: since nodes can and do overlap, there could be more than one leaf where the new point could be added.

This is such a big deal that we mentioned it as the second property determining SS-tree's shape: we need to choose a heuristic method to select which branch to traverse, or to select one of the leaves that would already contain the new point.

SS-trees originally used a simple heuristic: at each step, it will select the one branch whose centroid is closest to the point that is being inserted (those rare ties that will be faced can be broken arbitrarily).

**Listing 10.5 The searchParentLeaf method**

```
function searchParentLeaf(node,
target)                                          #1
  if node.leaf
then
#2
    return
node

  else
    child ←
node.findClosestChild(target)
    #3
    return searchParentLeaf(child,
target)                                      #4
```